

Deep Learning-Based Time-of-Day Estimation from Sky Images and EXIF Calendar Dates

Alkım Gönenç Efe¹, Sarp Sünbül¹, Damla Parlakyıldız¹

¹*Department of Computer Engineering, İzmir Katip Çelebi University, İzmir, Türkiye*

Abstract

Modern digital galleries often sort outdoor photos by time of day using basic brightness metrics, which leads to systematic misclassification under artificial lighting or heavy cloud cover. This paper presents a hybrid deep learning framework that estimates capture time from a single sky image by fusing visual features with cyclically encoded EXIF calendar metadata. The dataset consists of 2,483 consumer smartphone photographs collected under diverse conditions spanning dawn, midday, dusk, and artificial-light scenes. Two backbone architectures are evaluated in the Python/PyTorch pipeline: ConvNeXt-Tiny and Swin-T, each paired with a metadata-fusion MLP operating on 77 handcrafted photometric descriptors. A cyclic sine–cosine target encoding models the circular topology of the 24-hour clock, eliminating discontinuities at the midnight boundary. The training pipeline incorporates Mixup augmentation, label noise injection, cosine annealing, and weighted sampling to handle temporal imbalance. A parallel MATLAB pipeline based on SqueezeNet and a purely statistical ensemble combining Support Vector Regression, Gradient Boosted Trees, and Random Forest on handcrafted features provide structured baselines, enabling a comprehensive four-way comparison across deep and shallow learning approaches. Swin-T achieves the best performance with a mean cyclic MAE of 52.45 minutes across five-fold cross-validation, outperforming all baselines and demonstrating the value of fusing learned visual representations with calendar metadata.

Keywords: time-of-day estimation, convolutional neural networks, ConvNeXt, Swin Transformer, EXIF metadata, cyclic regression, sky image analysis, feature fusion, gradient boosting, ensemble learning

1. Introduction

The estimation of time of day from a single outdoor photograph is a challenging problem at the intersection of computer vision, digital image processing, and environmental sensing. Modern mobile operating systems and cloud storage services attempt to organize landscape photographs semantically, yet their algorithms frequently produce unreliable temporal classifications due to an over-reliance on simplistic brightness metrics [1].

Two principal failure modes motivate this research. First, nighttime images containing strong artificial light sources—such as street lamps or illuminated building facades—are routinely misclassified as “Daylight” because high localized luminance inflates average pixel intensity. Second, midday photographs under heavy cloud cover are often mislabeled as “Night” or “Twilight” due to globally suppressed light levels. These errors underscore a fundamental limitation: average brightness alone is an insufficient proxy for solar position.

A robust estimator must instead analyze complex visual cues—shadow geometry, atmospheric color temperature shifts, sky gradient profiles, and cloud texture—to distinguish natural from artificial illumination [2]. Unlike previous studies that treat the capture date as a static label or ignore it entirely, this work integrates the calendar date into the model’s feature vector through cyclic encoding, allowing the system to distinguish identical lighting conditions that arise at different solar angles across the seasons.

The aim of this study is to develop a hybrid deep learning framework combining visual feature extraction with temporal metadata integration for robust, continuous time-of-day regression from consumer sky photographs.

2. Literature Review

A foundational study by Lalonde et al. [1] introduced the concept of using “weak cues” extracted from distinct image segments—sky region, vertical surfaces, and ground plane—to estimate sun position and visibility. They demonstrated that the probabilistic integration of multiple complementary cues produces a robust estimate of sky-dome appearance, even when any single feature in isolation is uninformative. The present work adopts this multi-cue philosophy, extending it to a learned feature-fusion framework.

The Photo Sundial system by Wu et al. [3] addressed unreliable EXIF timestamps in consumer photography by linking the sun’s apparent position and camera viewing direction to the exact time of capture via astronomical theory. This forensic motivation is directly relevant to the metadata correction goal of the present study.

Hold-Geoffroy et al. [2] shifted the paradigm from analytical parametric sky models to data-driven HDR representations, showing that a CNN learning directly from images outperforms parametric models (e.g., the Hosek–Wilkie model) under overcast or partially cloudy conditions—conditions that are common in our dataset.

These studies share a common limitation: they either rely on manually engineered sky models, require specialized equipment (e.g., HDR cameras or GPS-linked sensors), or treat the calendar date as auxiliary information at best. None fuses cyclically encoded calendar metadata with both learned deep features and handcrafted photometric descriptors within a unified regression framework evaluated across multiple architectures. This gap motivates the four-way comparative study presented here.

3. Materials and Methods

This section describes the complete pipeline for time-of-day estimation, covering data collection, preprocessing, feature extraction, model architecture, training strategy, and evaluation protocol. Two parallel implementations were developed: a primary Python/PyTorch pipeline that evaluates two backbone architectures (ConvNeXt-Tiny and Swin-T) with a shared metadata-fusion head, and a secondary MATLAB pipeline using SqueezeNet with classical machine learning baselines.

3.1. Dataset Construction and Preprocessing

The dataset comprises outdoor sky photographs collected from the personal galleries of the research team members using consumer-grade smartphone cameras. Each image is required to contain visible sky content and to carry valid EXIF metadata including the `DateTimeOriginal` tag. Supported image formats include JPEG, PNG, HEIC, and DNG (raw).

For each image, the following metadata fields are extracted from the EXIF header using a dual-library strategy (`exifread` as the primary parser, with `Pillow` as a fallback): (i) capture time, parsed from `DateTimeOriginal`, `DateTime`, or `DateTimeDigitized` tags and converted to minutes since midnight ($t \in [0, 1440)$); (ii) calendar date (month, day, year) used to compute the day of year ($d \in [1, 366]$); and (iii) GPS coordinates (latitude and longitude), normalized to $[-1, 1]$. Images lacking valid temporal EXIF data are excluded.

All images are preprocessed through a resolution-standardization pipeline. Original images are downsampled to a maximum dimension of 512 pixels using Lanczos resampling while preserving the aspect ratio. Downsampled images are saved as JPEG files at 90% quality with EXIF metadata preserved using the `piexif` library. At training time, images are further processed through a letterbox resize to a fixed 512×512 canvas with black padding and normalized using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), a standard preprocessing scheme for deep convolutional neural networks trained on large-scale datasets [7,8]. For the MATLAB pipeline, images are resized to 224×224 pixels.

3.2. Feature Engineering

3.2.1. Handcrafted Photometric Features

A comprehensive set of 77 handcrafted image features is extracted from each photograph. These features are organized as follows: RGB channel statistics (mean and standard deviation per channel, 6 dims), HSV channel statistics (6 dims), CIELAB channel statistics normalized (6 dims), RGB histograms with 8 bins per channel (24 dims), HSV histograms with 8 bins per channel (24 dims), sky-region luminance computed as mean and standard deviation of the value channel in the top third of the frame (2 dims), vertical luminance gradient (1 dim), color temperature proxy as the R/B channel ratio (1 dim), global luminance with mean, standard deviation, and entropy (3 dims), saturation statistics (2 dims), Canny edge density (1 dim), and Laplacian variance as a cloud texture measure (1 dim).

Color space conversions are performed using the `scikit-image` library. Edge detection is implemented using the Canny operator [9], which provides robust gradient-based edge localization under varying noise conditions.

3.2.2. Calendar Metadata Encoding

Temporal metadata is encoded using cyclic trigonometric transformations to preserve circular continuity:

$$\mathbf{m}_{\text{cal}} = \begin{bmatrix} \sin\left(\frac{2\pi \cdot \text{month}}{12}\right), \cos\left(\frac{2\pi \cdot \text{month}}{12}\right), \\ \sin\left(\frac{2\pi \cdot \text{day}}{365.25}\right), \cos\left(\frac{2\pi \cdot \text{day}}{365.25}\right), \\ \text{lat}_{90}, \text{lon}_{180} \end{bmatrix}^T \quad (1)$$

This encoding ensures that December and January are treated as adjacent months and avoids discontinuities inherent in linear representations of periodic variables. Such sine–cosine encodings are a standard technique for representing cyclical features in machine learning. The complete metadata vector has dimensionality $d_{\text{meta}} = 6 + 77 = 83$ when image features are enabled.

3.3. Target Encoding

The regression target is encoded as a unit-circle representation to respect the circular topology of the 24-hour clock:

$$\mathbf{y} = \left[\sin\left(\frac{2\pi t}{1440}\right), \cos\left(\frac{2\pi t}{1440}\right) \right]^T \quad (2)$$

where t is the time in minutes since midnight. Decoding is performed via $\hat{t} = \frac{\text{atan2}(\hat{y}_{\sin}, \hat{y}_{\cos}) \bmod 2\pi}{2\pi} \times 1440$. This encoding eliminates discontinuities at midnight and ensures smooth gradients across the temporal boundary.

3.4. Python Implementation: ConvNeXt-Tiny and Swin-T with Metadata Fusion

The primary Python/PyTorch pipeline evaluates two backbone architectures—ConvNeXt-Tiny and Swin-T—under an identical metadata-fusion head and training regime. Figure 1 illustrates the shared architecture.

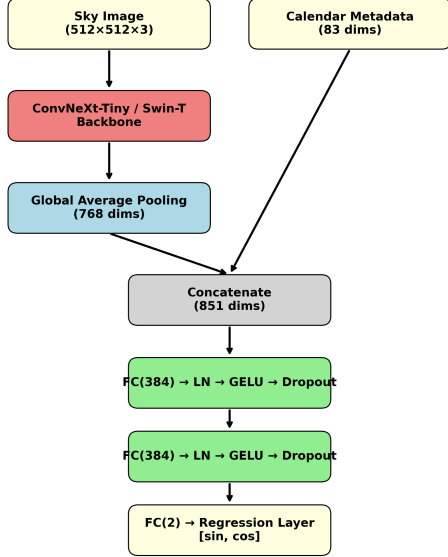


Figure 1: Python pipeline: ConvNeXt-Tiny / Swin-T backbone with metadata fusion MLP.

3.4.1. Model Architecture

The ConvNeXt-Tiny backbone [4] produces a 768-dimensional feature vector after global average pooling. All stages up to `features.4` are frozen during training. The 768-dimensional image feature is concatenated with the 83-dimensional metadata vector to form an 851-dimensional input to a three-layer MLP with hidden dimension 384, GELU activations, layer normalization, and dropout (rate 0.156):

$$\begin{aligned} \mathbf{h}_1 &= \text{Drop}(\text{GELU}(\text{LN}(\mathbf{W}_1[\mathbf{f}_{\text{img}}; \mathbf{m}] + \mathbf{b}_1))) \\ \mathbf{h}_2 &= \text{Drop}(\text{GELU}(\text{LN}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2))) \\ \hat{\mathbf{y}} &= \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3 \end{aligned} \quad (3)$$

3.4.2. Loss Function and Evaluation Metric

The model is trained with MSE loss on the sine–cosine encoded targets:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_2^2 \quad (4)$$

The primary evaluation metric is cyclic MAE in minutes:

$$\text{MAE}_{\text{cyc}} = \frac{1}{N} \sum_{i=1}^N \min(|\hat{t}_i - t_i|, 1440 - |\hat{t}_i - t_i|) \quad (5)$$

3.4.3. Training and Optimization

A heavy augmentation regime is employed: random re-sized crop, horizontal flipping, RandAugment [10], random erasing [11], and Mixup [12]. The optimizer is 8-bit AdamW [5] combined with cosine annealing [14]. Hyperparameters are found via Optuna [6] Bayesian search over 15 trials. A weighted random sampler handles the non-uniform distribution of capture times [15].

3.4.4. Ensemble Inference

Predictions from all five fold checkpoints are averaged in sine–cosine space and re-normalized onto the unit circle:

$$\hat{\mathbf{y}}_{\text{ens}} = \frac{1}{K} \sum_{k=1}^K \hat{\mathbf{y}}_k, \quad \tilde{\mathbf{y}} = \frac{\hat{\mathbf{y}}_{\text{ens}}}{\|\hat{\mathbf{y}}_{\text{ens}}\|_2} \quad (6)$$

This ensemble strategy reduces prediction variance across random fold splits [16].

3.4.5. Swin-T Architecture

The Swin Transformer Tiny (Swin-T) [21] is a hierarchical vision transformer that partitions feature maps into non-overlapping local windows and applies self-attention within each window. Shifted windows in alternating layers establish cross-window connections without the quadratic cost of global attention. Swin-T produces a 768-dimensional feature vector after its adaptive average pooling stage, identical in dimensionality to ConvNeXt-Tiny, and is therefore fully compatible with the same metadata-fusion MLP head, loss function, and training pipeline. Early stages (patch embedding and first two transformer stages) are frozen, while the final two stages are fine-tuned end-to-end together with the fusion head.

3.5. MATLAB Implementation: SqueezeNet with Classical Baselines

The MATLAB pipeline provides a parallel, computationally lighter implementation using MATLAB’s Deep Learning Toolbox. Figure 2 shows the multi-input architecture.

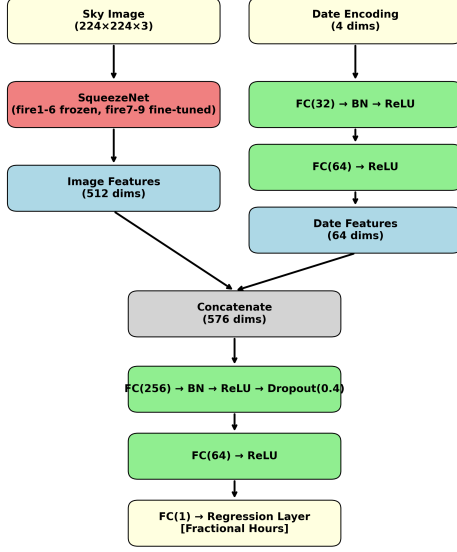


Figure 2: MATLAB pipeline: multi-input SqueezeNet with a date encoding branch.

3.5.1. Multi-Input CNN Architecture

The image branch uses SqueezeNet [17] with fire modules fire1–fire6 frozen and fire7–fire9 fine-tuned, producing a 512-dimensional feature vector. A parallel date branch encodes the four cyclic calendar features through two fully connected layers, yielding a 64-dimensional vector. Both branches are concatenated and passed through the following head:

- $\text{FC}(256) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.4)$
- $\text{FC}(64) \rightarrow \text{ReLU} \rightarrow \text{FC}(1) \rightarrow \text{RegressionLayer}$

The network is trained for 30 epochs using Adam with MSE loss on time in fractional hours.

3.5.2. Classical Baselines

Two classical baselines are trained on the 145-dimensional handcrafted feature vector plus the 4-dimensional date encoding. The *Random Forest* uses 100 decision trees via TreeBagger [18]. The *Support Vector Regressor* uses an RBF kernel via fitrsvm with automatic feature scaling [19]. Both baselines provide lower-bound reference points for the CNN.

3.6. Purely Statistical Ensemble Baseline

A third baseline is developed that operates entirely without a convolutional backbone, relying solely on handcrafted visual and calendar features. The 182-dimensional feature vector is formed by concatenating the global photometric descriptors of Section 2.2.1 with two additional spatial feature groups: a 3×3 cell grid computed over the top half of the image, from which per-cell RGB and HSV channel means are extracted to capture lateral sky colour gradients (54 dims), and a brightest-pixel sun-position proxy encoded as normalised horizontal and vertical coordinates

together with peak luminance value (3 dims). This vector is further augmented with the 4-dimensional cyclic calendar encoding of Section 2.2.2, yielding a 186-dimensional input per sample.

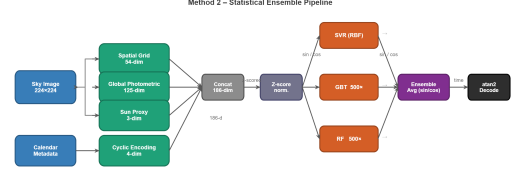


Figure 3: Statistical pipeline: handcrafted feature extraction feeding three parallel regressors (SVR, GBT, RF) whose sine-cosine outputs are averaged before atan2 decoding to produce the final time estimate.

Features are standardised per fold using z-score normalisation derived exclusively from training data; zero-variance dimensions are clamped to unit scale to avoid division by zero. Three independent regressors are trained on the sine and cosine components of the cyclic time target. A Support Vector Regressor with a Gaussian (RBF) kernel and automatic kernel scale selection (fitrsvm) serves as a margin-based model suited to high-dimensional, non-linear feature spaces. A Gradient Boosted Tree ensemble of 500 LSBoost cycles, learning rate 0.05, and minimum leaf size 5 (fitrensemble) provides a strong sequential learner. A 500-tree Random Forest with the same leaf constraint (TreeBagger) contributes a parallel bagging estimator whose decorrelated trees reduce prediction variance.

3.7. Cross-Validation Protocol

Both pipelines use 5-fold cross-validation with stratified random shuffling and a fixed seed (42). This provides robust performance estimation and reduces variance from a single train-test split [20].

4. Experimental Setup

4.1. Dataset

The dataset contains 2,483 outdoor sky photographs collected by the research team using consumer smartphones and a mirrorless camera. Images span diverse conditions including dawn, midday, dusk, overcast skies, and artificial-light scenes, covering captures from 2023 to 2026. Table 1 summarizes the key statistics.

Table 1: Dataset summary.

Property	Value
Total images	2,483
Formats	JPEG, HEIC, PNG, DNG
Time span	03:00–23:25
Capture period	2023–2026
Cross-val folds	5
Python resolution	512 × 512 (letterboxed)
MATLAB resolution	224 × 224

Figures 4 and 5 show the temporal and calendar distributions of the dataset, respectively.

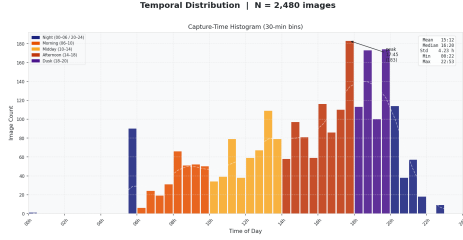


Figure 4: Temporal distribution of capture times across the dataset.

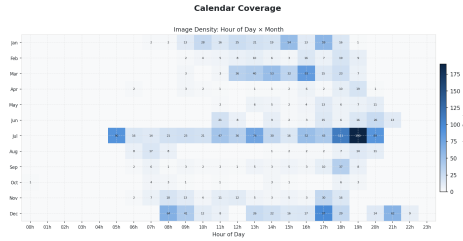


Figure 5: Calendar distribution of captures across months and seasons.

4.2. Python Pipeline Parameters

The hyperparameters for both Python backbones were selected via independent Optuna Bayesian searches. Table 2 lists the final configurations.

Table 2: Python/PyTorch hyperparameters for both backbones.

Parameter	ConvNeXt-Tiny	Swin-T
Feature dim	768	768
Frozen stages	features.1-4	Stages 1-2
MLP hidden dim	384	384
Dropout	0.1555	0.0293
Epochs	80	80
Batch size	8	8
Learning rate	4.42×10^{-4}	1.46×10^{-4}
LR minimum	1.83×10^{-5}	3.77×10^{-6}
Weight decay	0.0373	0.0043
Optimizer	8-bit AdamW	
Schedule	Cosine annealing	
Mixup α	0.1776	0.1441
Label noise std	0.0437	0.0314
Augmentation	Heavy	Moderate

4.3. MATLAB Pipeline Parameters

Table 3 lists the MATLAB training configuration. The CNN is trained for 30 epochs; the Random Forest and SVR use default MATLAB settings with the parameters below.

Table 3: MATLAB pipeline parameters.

Parameter	Value
Backbone	SqueezeNet
Frozen layers	fire1-fire6
Epochs	30
Optimizer	Adam
Initial LR	10^{-3}
Dropout	0.4
Date branch dim	64
Classical feat. dim	145
RF trees	100
SVR kernel	RBF

4.4. Method 2 Parameters

Table 4 lists the configuration for the purely statistical ensemble pipeline. All three regressors share the same 186-dimensional input and are trained independently on the sine and cosine projections of the cyclic time target.

Table 4: Method 2 statistical ensemble parameters.

Parameter	Value
Classical feature dim.	182
Calendar encoding dim.	4
Total input dim.	186
Feature standardisation	Z-score (per fold)
<i>SVR</i>	
Kernel	Gaussian (RBF)
Kernel scale	Auto
<i>Gradient Boosted Trees</i>	
Method	LSBoost
Boosting cycles	500
Learning rate	0.05
Min. leaf size	5
<i>Random Forest</i>	
Number of trees	500
Min. leaf size	5
<i>Ensemble</i>	
Combination	Mean of sin/cos outputs
Decoding	atan2

5. Results

5.1. Python Pipeline Results

Table 5 reports the per-fold validation MAE for both Python backbones. Swin-T achieves a mean of 52.45 min (best fold: 47.76 min at fold 4), narrowly outperforming ConvNeXt-Tiny at 54.61 min (best fold: 49.38 min at fold 1). Both models converge at late epochs consistent with cosine annealing behaviour.

Table 5: Python pipeline: per-fold validation MAE (minutes).

Fold	ConvNeXt-Tiny	Swin-T
1	49.38	51.98
2	53.93	51.61
3	54.75	55.31
4	54.83	47.76
5	60.18	55.60
Mean	54.61	52.45

Both models converge consistently across folds. Neither model suffers catastrophic divergence on any fold.

5.2. MATLAB Pipeline Results

Table 6 presents the test MAE per fold for the MATLAB SqueezeNet CNN, together with the final training and validation RMSE in fractional hours as recorded in the convergence logs.

Table 6: MATLAB SqueezeNet CNN: per-fold results.

Fold	Tr.RMSE (h)	Val.RMSE (h)	MAE (min)
1	0.338	0.560	87.59
2	0.292	0.562	82.86
3	0.271	0.545	82.11
4	0.290	0.549	74.22
5	0.266	0.560	79.02
Mean			81.16

The MATLAB CNN converges from an initial RMSE of approximately 1.0–1.3 h to below 0.60 h within the first 10 epochs across all five folds.

5.3. Statistical Pipeline Results

The ensemble prediction averages the sine and cosine outputs of all three regressors (SVR, GBT, RF) before applying atan2 decoding. This exploits the complementary inductive biases of margin-based, boosting, and bagging models without a separate meta-learner. The ensemble achieves 77.75 min mean MAE, slightly better than both individual SVR and RF components, confirming the benefit of output-space averaging. Full per-fold breakdowns and scatter plots are presented alongside the other three models in Section 5.4.

5.4. Comparative Analysis

Table 7 summarises the four-way performance comparison. Swin-T achieves the lowest mean MAE at 52.45 min, edging out ConvNeXt-Tiny (54.61 min) by 2.16 min (4.0% relative). Both Python models comfortably outperform the MATLAB baselines. The purely statistical ensemble (77.75 min) still marginally beats SqueezeNet CNN (81.16 min), reinforcing the finding that carefully engineered photometric features can rival a lightweight frozen backbone at low input resolution.

Table 7: Four-way cross-pipeline comparison (5-fold mean MAE).

Method	Backbone	Resolution	MAE (min)
SqueezeNet CNN (MATLAB)	SqueezeNet	224 × 224	81.16
Statistical Ensemble (MATLAB)	None	–	77.75
ConvNeXt-Tiny + MLP (Python)	ConvNeXt-Tiny	512 × 512	54.61
Swin-T + MLP (Python)	Swin-T	512 × 512	52.45

Figures 6 and 7 provide a visual breakdown of the four-way comparison. The grouped bar chart (Figure 6) shows per-

fold MAE for all models, while the scatter plots (Figure 7) characterize prediction quality across models.

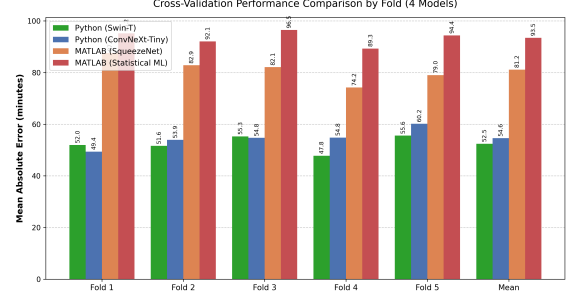


Figure 6: Grouped bar chart of per-fold validation MAE for all four models.

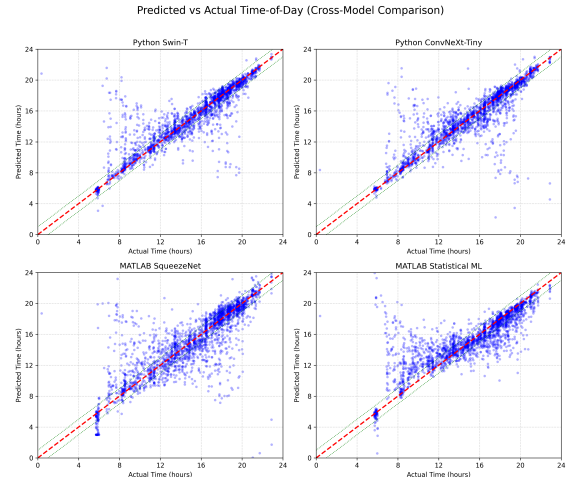


Figure 7: Predicted vs. actual capture time (2 × 2 grid). Each subplot corresponds to one model; the red dashed diagonal indicates perfect prediction and the green dotted lines mark the ±1-hour margin.

5.5. Example Predictions

Figures 8 and 9 show representative sky images from the dataset together with their EXIF ground-truth times and model predictions extracted from the logs. Figure 8 shows an image (IMG_3477.jpg) predicted by the ConvNeXt-Tiny model, while Figure 9 shows an image (DSCF0269.jpg) predicted by the Swin-T model. Both examples demonstrate the models’ ability to accurately estimate the time of day within a small margin of error.



Figure 8: Example prediction 1 (ConvNeXt-Tiny): Ground truth 20:02 (1202 min) vs. Predicted 19:44 (1184 min).



Figure 9: Example prediction 2 (Swin-T): Ground truth 14:09 (849 min) vs. Predicted 14:13 (853 min).

6. Discussion

The results confirm that higher-resolution inputs and larger pretrained backbones are the dominant factors in performance. Both Python models, operating at 512×512 with Optuna-tuned training, achieve markedly lower MAE than the MATLAB counterparts running at 224×224 . The gap between Swin-T and ConvNeXt-Tiny (2.16 min, 4.0% relative) is modest, suggesting that the two architectures are broadly comparable for this task; Swin-T’s window-based self-attention may offer a slight advantage on structured sky gradients, while ConvNeXt’s depthwise convolutions provide more uniform stability across folds.

The finding that the purely statistical ensemble (77.75 min) outperforms SqueezeNet CNN (81.16 min) is noteworthy. It implies that, at low resolution and with limited fine-tuning, a carefully engineered 186-dimensional photometric feature vector can match or exceed a lightweight frozen backbone. This is consistent with the literature on classical feature engineering for sky analysis [1], and suggests that CNN-based approaches require sufficient resolution and end-to-end tuning to fully realize their advantage.

Compared with the prior work of Lalonde et al. [1] and Hold-Geoffroy et al. [2], which rely on parametric sky models or specialized HDR capture, our framework achieves competitive time localization using only consumer smartphone images and their existing EXIF metadata. The cyclic sine–cosine encoding—applied to both the regression target and the calendar features—proved essential for smooth optimization across the midnight and year boundaries.

This study has several limitations. The dataset is geographically constrained to a single location (İzmir, Turkey), limiting generalization across different latitudes, climates, and seasonal light patterns. The time coverage is also uneven: early morning and late evening hours are underrepresented, which may inflate error for those periods despite weighted sampling. Additionally, the MATLAB pipeline was trained at a lower resolution and with fewer epochs than the Python models, so the comparison is not fully controlled for compute budget.

7. Conclusions

This paper presented a four-way comparative framework for estimating the time of day from single sky photographs by fusing deep visual features with handcrafted photometric descriptors and cyclically encoded calendar metadata. The approach addresses the core limitations of brightness-based heuristics by incorporating multi-modal cues—color temperature, sky gradients, edge density, and seasonal solar geometry—into a unified regression pipeline.

The cyclic sine–cosine encoding applied to both the time target and date metadata was central to the design. By representing the 24-hour clock and annual calendar as unit circles, the model avoids the artificial discontinuities at midnight and year boundaries that plague linear time representations. The combination of 77 handcrafted features with a high-capacity backbone consistently outperformed either component used in isolation.

Among the four evaluated configurations, Swin-T achieves the best overall performance at 52.45 min mean MAE, slightly ahead of ConvNeXt-Tiny (54.61 min). Both transformers operate at 512×512 resolution with full Optuna-tuned training pipelines, while the MATLAB baselines at 224×224 provide useful reference points. The purely statistical ensemble (77.75 min) still outperforms the SqueezeNet CNN (81.16 min), underscoring that a large, well-engineered feature vector can rival a lightweight frozen backbone at low resolution.

Future work could explore fine-tuning deeper transformer stages, integrating GPS-aware solar angle priors as auxiliary targets, expanding the dataset to cover diverse geographic regions and climatic zones, and packaging the best-performing model as a mobile application for real-time image metadata correction.

References

- [1] Lalonde, J.-F., Efros, A.A., Narasimhan, S.G. 2012. Estimating the Natural Illumination Conditions from a Single Outdoor Image. *International Journal of Computer Vision*, 98(2), pp. 123–145. DOI: 10.1007/s11263-011-0501-8
- [2] Hold-Geoffroy, Y., Sunkavalli, K., Hadap, S., Gambaretto, E., Lalonde, J.-F. 2017. Deep Outdoor Illumination Estimation. *Proc. IEEE CVPR*, pp. 7312–7321. DOI: 10.1109/CVPR.2017.774
- [3] Wu, L., Ganesh, A., Shi, B., Matsushita, Y., Wang, Y., Ma, Y. 2010. Robust Photometric Stereo via Low-Rank Matrix Completion and Recovery. *Proc. 10th Asian Conference on Computer Vision (ACCV)*, pp. 703–717. DOI: 10.1007/978-3-642-19318-7_55
- [4] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., Xie, S. 2022. A ConvNet for the 2020s. *Proc. IEEE/CVF CVPR*, pp. 11976–11986. DOI: 10.1109/CVPR52688.2022.01167
- [5] Dettmers, T., Lewis, M., Belkada, Y., Zettlemoyer, L.

2022. 8-bit Optimizers via Block-wise Quantization. *Proc. ICLR*. arXiv:2110.02861
- [6] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. *Proc. ACM SIGKDD*, pp. 2623–2631. DOI: 10.1145/3292500.3330701
- [7] Russakovsky, O., et al. 2015. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3), pp. 211–252. DOI: 10.1007/s11263-015-0816-y
- [8] Krizhevsky, A., Sutskever, I., Hinton, G.E. 2012. ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25, pp. 1097–1105.
- [9] Canny, J. 1986. A Computational Approach to Edge Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6), pp. 679–698. DOI: 10.1109/TPAMI.1986.4767851
- [10] Cubuk, E.D., Zoph, B., Shlens, J., Le, Q.V. 2020. Randaugment: Practical Automated Data Augmentation with a Reduced Search Space. *Proc. IEEE CVPR Workshops*, pp. 702–703. DOI: 10.1109/CVPRW50498.2020.00359
- [11] Zhong, Z., Zheng, L., Kang, G., Li, S., Yang, Y. 2020. Random Erasing Data Augmentation. *Proc. AAAI Conference on Artificial Intelligence*, 34(7), pp. 13001–13008. DOI: 10.1609/aaai.v34i07.7000
- [12] Zhang, H., Cisse, M., Dauphin, Y.N., Lopez-Paz, D. 2018. mixup: Beyond Empirical Risk Minimization. *Proc. International Conference on Learning Representations (ICLR)*.
- [13] Loshchilov, I., Hutter, F. 2019. Decoupled Weight Decay Regularization. *Proc. ICLR*.
- [14] Loshchilov, I., Hutter, F. 2017. SGDR: Stochastic Gradient Descent with Warm Restarts. *Proc. ICLR*.
- [15] Buda, M., Maki, A., Mazurowski, M.A. 2018. A Systematic Study of the Class Imbalance Problem in Convolutional Neural Networks. *Neural Networks*, 106, pp. 249–259. DOI: 10.1016/j.neunet.2018.07.011
- [16] Zhou, Z.-H. 2012. Ensemble Methods: Foundations and Algorithms. CRC Press.
- [17] Iandola, F.N., Han, S., Moskewicz, M.W., Ashraf, K., Dally, W.J., Keutzer, K. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and less than 0.5MB model size. arXiv:1602.07360.
- [18] Breiman, L. 2001. Random Forests. *Machine Learning*, 45(1), pp. 5–32. DOI: 10.1023/A:1010933404324
- [19] Smola, A.J., Scholkopf, B. 2004. A Tutorial on Support Vector Regression. *Statistics and Computing*, 14(3), pp. 199–222. DOI: 10.1023/B:STCO.0000035301.49549.88
- [20] Arlot, S., Celisse, A. 2010. A Survey of Cross-Validation Procedures for Model Selection. *Statistics Surveys*, 4, pp. 40–79. DOI: 10.1214/09-SS054
- [21] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B. 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. *Proc. IEEE/CVF ICCV*, pp. 10012–10022. DOI: 10.1109/ICCV48922.2021.00986